# 格式化字符串基础(x86)

## CONTENT


- 原理
- printf()函数的特性
  - 特性一：printf()函数的参数个数不固定
  - 特性二：利用%n格式符写入数据
  - 特性三：自定义打印字符串宽度


## 原理

格式化字符串漏洞和普通的栈溢出有相似之处，print()、fprint()等*print()系列的函数可以按照一定的格式将数据进行输出

```c
printf("Welcome to %s", "my blog")
```

该printf函数的第一个参数就是格式化字符串，它来告诉程序以什么格式输出printf()函数的一般形式为：printf("format"，输出表列)，format结构如下：

```
%[标志][输出最小宽度][.精度][长度]类型
```

其中，和格式化字符串漏洞有关系的主要为如下几点：

1. 输出最小宽度：用十进制整数来表示输出的最少位数。若实际位数多于定义的宽度，则按实际位数输出；若实际位数少于定义的宽度则补空格或0
2. 类型

> **Info**
>
> - %c：输出单个字符，配上%n可用于向指定地址写入数据
> - %d：输出十进制整数，配上%n可用于向指定地址写入数据
> - %x：输出十六进制数据，如%i$x表示要泄露偏移i处4字节长的16进制数据，%i$lx表示要泄露偏移i处8字节长的16进制数据，32bit和64bit环境下一样
> - %p：输出十六进制数据，与%x基本一致，只是附加了前缀0x，在32bit下输出4字节，在64bit下输出8字节，可通过输出字节长度来判断目标环境是

32bit还是64bit

- %s：输出内容为字符串，即将偏移处指针指向的字符串输出，如%i$s表示输出偏移i处地址所指向的字符串，在32bit和64bit环境下一样，可用于读取GOT表等信息
- %n：将%n之前printf已经打印的字符个数赋值给偏移处指针所指向的地址位置，如%100x10$n表示将0x64写入偏移10处保存的指针所指向的地址（4字节），而%hn表示写入的地址空间为2字节，%hhn表示写入的地址空间为1字节，%lln表示写入的地址空间为8字节，在32bit和64bit环境下一样。不过，有时直接写入4字节会导致程序崩溃或等待时间过长，可以通过%$hn或%$hhn来适时调整

*%n是通过格式化字符串漏洞改变程序流程的关键方式，而其他格式化字符串参数可用于读取信息或配合%n写入数据*

当我们正常使用printf()函数时：

```c
char str[100];
scanf("%s", str);
printf("%s", str);
```

但是如果出现如下情况时：

```c
char str[100];
scanf("%s", str);
printf(str);
```

很明显，第二个程序中的printf()函数的参数我们是可控的，我们在控制了format参数后结合printf()函数的特性就可以进行对应的攻击

# printf()函数的特性

## 特性一：printf()函数的参数个数不固定

我们可以利用这一特性进行越界数据的访问，下面是一个正常的程序：

```c
#include <stdio.h>

int main(void)
{
    int a = 1, b = 2, c = 3;
    char buf[] = "test";

    printf("%s %d %d %d\n", buf, a, b, c);

    return 0;
}
```

我们编译之后运行测试一下：

```shell
bh@Mandarava23:/mnt/d/pwn/learning/format_string$ gcc -m32 -fno-stack-protector -o format format.c
bh@Mandarava23:/mnt/d/pwn/learning/format_string$ ./format
test 1 2 3
```

然后我们再增加一个printf函数的format参数，变为以下内容：

```c
printf("%s %d %d %d %x\n", buf, a, b, c);
```

一样编译运行测试：

```shell
bh@Mandarava23:/mnt/d/pwn/learning/format_string$ gcc -m32 -z execstack -fno-stack-protector -o format1 format.c
format.c: In function ‘main’:
format.c:8:26: warning: format ‘%x’ expects a matching ‘unsigned int’ argument [-Wformat=]
    8 |     printf("%s %d %d %d %x\n", buf, a, b, c);
      |                         ~^
      |                          |
      |                          unsigned int
bh@Mandarava23:/mnt/d/pwn/learning/format_string$ ./format1
test 1 2 3 f7f56000
```

可以发现，gcc在编译时警告我们，多了一个format参数，但最后还是编译通过了，而且结果多输出了一个f7f56000，这是什么呢？我们可以通过gdb来动调一下看看，在printf函数这里下一个断点，然后运行，程序停在了printf函数的入口处

```
0xf7dd9120 <printf>       call   0xf7ef844d
```

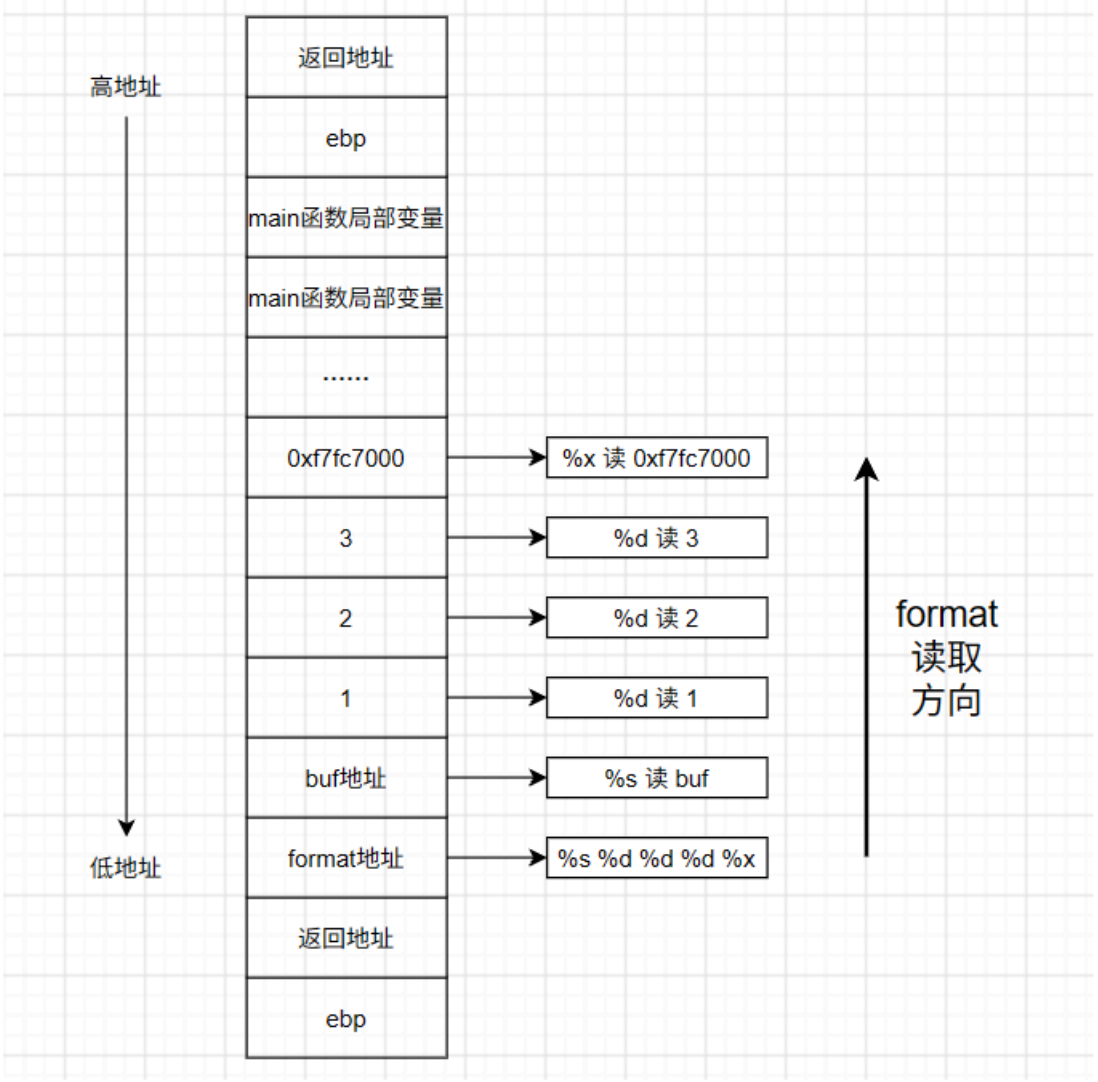
我们来看一下此时的栈布局：

```
pwndbg> telescope $sp 10
00:0000│ esp 0xffffcf9c —▸ 0x565561e7 (main+90) ◂— add esp, 0x20
01:0004│-048 0xffffcfa0 —▸ 0x56557008 ◂— '%s %d %d %d %x\n'
02:0008│-044 0xffffcfa4 —▸ 0xffffcfcf ◂— 'test'
03:000c│-040 0xffffcfa8 ◂— 1
04:0010│-03c 0xffffcfac ◂— 2
05:0014│-038 0xffffcfb0 ◂— 3
06:0018│-034 0xffffcfb4 —▸ 0xf7fc7000 ◂— jg 0xf7fc7047
07:001c│-030 0xffffcfb8 ◂— 0
08:0020│-02c 0xffffcfbc —▸ 0x565561a4 (main+23) ◂— add eax, 0x2e34
09:0024│-028 0xffffcfc0 ◂— 0
```

可以看到，这里的esp+18指向的值是0xf7fc7000，这里其实就是%x读取的内容
诶，居然不是上面打印出来f7f56000吗？难道是gdb动调出了什么问题？其实不是的，因为上面的f7f56000是另一次运行的残留值。怎么证明呢？我们不妨让程序继续运行，再把%x输出出来：

```
pwndbg> telescope $sp 10
00:0000│ esp 0xffffcf9c —▸ 0x565561e7 (main+90) ◂— add esp, 0x20
01:0004│-048 0xffffcfa0 —▸ 0x56557008 ◂— '%s %d %d %d %x\n'
02:0008│-044 0xffffcfa4 —▸ 0xffffcfcf ◂— 'test'
03:000c│-040 0xffffcfa8 ◂— 1
04:0010│-03c 0xffffcfac ◂— 2
05:0014│-038 0xffffcfb0 ◂— 3
06:0018│-034 0xffffcfb4 —▸ 0xf7fc7000 ◂— jg 0xf7fc7047
07:001c│-030 0xffffcfb8 ◂— 0
08:0020│-02c 0xffffcfbc —▸ 0x565561a4 (main+23) ◂— add eax, 0x2e34
09:0024│-028 0xffffcfc0 ◂— 0
pwndbg> c
Continuing.
test 1 2 3 f7fc7000
[Inferior 1 (process 7521) exited normally]
```

可以发现，现在这个进程中我们继续运行程序，打印出来的值就是我们动调分析出来的0xf7fc7000了。所以调用printf()函数后的栈布局也显而易见了



因此，我们只要能够控制format，就可以一直读取内存数据

```c
printf("%s %d %d %d %x %x %x %x %x\n", buf, a, b, c);
```

```shell
bh@Mandarava23:/mnt/d/pwn/learning/format_string$ ./format1
test 1 2 3 f7ef0000 0 565f01a4 0 0
```

这个例子也只是告诉我们能够利用%x一直读取站内的内存数据，但是这并不能满足我们的需求，我们要的是任意地址读取，这也是可以做到的，分析以下例子：

```c
#include <stdio.h>

int main(int argc, char *argv[])
{
    char str[200];

    fgets(str, 200, stdin);
    printf(str);

    return 0;
}
```

经过了第一个例子的讲解，我们可以直接尝试去读取str[]的内容：
gdb调试，单步运行完0x565561d5 call fgets@plt后输入：

```
aaaa%08x%08x%08x%08x%08x%08x
```

接着，我们执行到printf()函数，观察此时的栈区，特别注意 `0xffffcef8` 这个str数组的起始地址：

```
pwndbg> telescope $sp 10
00:0000| esp 0xffffcee0 —▸ 0xffffcef8 ◂— 'aaaa%08x%08x%08x%08x%08x%08x\n'
01:0004|-0e4 0xffffcee4 ◂— 0xc8
02:0008|-0e0 0xffffcee8 —▸ 0xf7fab5c0 (_IO_2_1_stdin_) ◂— 0xfbad2288
03:000c|-0dc 0xffffceec —▸ 0x565561b7 (main+26) ◂— add ebx, 0x2e19
04:0010|-0d8 0xffffcef0 —▸ 0x56555034 ◂— 6
05:0014|-0d4 0xffffcef4 ◂— 0xb /* '\x0b' */
06:0018| eax 0xffffcef8 ◂— 'aaaa%08x%08x%08x%08x%08x%08x\n'
07:001c|-0cc 0xffffcefc ◂— '%08x%08x%08x%08x%08x%08x\n'
... ↓        2 skipped
```

继续执行，看我们能够获得什么：

```
aaaa000000c8f7fab5c0565561b7565550340000000b61616161
```

可以看见，我们成功地读到了aaaa，即末尾的61616161。这时候我们需要借助printf()函数的另一个重要格式化字符参数%s，我们可以利用%s来获取指针指向的内存数据，那么我们便可以构造一个format尝试去获取0xffffcef8这个地址上的数据：

```
\xf8\xce\xff\xff%08x%08x%08x%08x%08x%s
```

如此我们便可以成功做到了任意地址读取，我们现在也只是能读个数据而已，我们能不能利用这个漏洞修改内存信息从而劫持程序执行流程呢，这就需要我们来看printf()函数的第二个特性了

## 特性二：利用%n格式符写入数据

%n是一个不经常用到的格式符，它的作用是把前面已经打印的长度写入某个内存地址，来看以下代码：

```c
#include <stdio.h>

int main()
{
    int num = 66666666;

    printf("Before: num = %d\n", num);
    printf("%d%n\n", num, &num);
    printf("After: num = %d\n", num);
}
```

可以发现我们用%n成功修改了num的值：

```
Before: num = 66666666
66666666
After: num = 8
```

现在我们已经知道可以用构造的格式化字符串去访问栈内的数据，并且可以利用%n向内存中写入值，那我们是不是可以修改某一个函数的返回地址从而控制程序执行流程呢，我们可以发现，%n的作用只是将前面打印的字符串长度写入到内存中，而我们想要写入的是一个地址，而且这个地址是很大的。这时候我们就需要用到printf()函数的第三个特性来配合完成地址的写入

## 特性三：自定义打印字符串宽度

在格式符中间加上一个十进制整数来表示输出的最少位数，若实际位数多于定义的宽度，则按实际位数输出，若实际位数少于定义的宽度则补以空格或0：

```c
int main()
{
    int num = 66666666;

    printf("Before: num = %d\n", num);
    printf("%.100d%n\n", num, &num);
    printf("After: num = %d\n", num);
}
```

```
Before: num = 66666666
00000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000066666666
After: num = 100
```

可以看到，num的值已经改为了100。因此我们要将地址写入内存时，就可以把地址对应的十进制数作为格式控制符来控制宽度：

```c
printf("%.16666666d%n\n", num, &num);
printf("After: num = %z\n", num);
```

可以看到，num已经被成功修改为fe502a